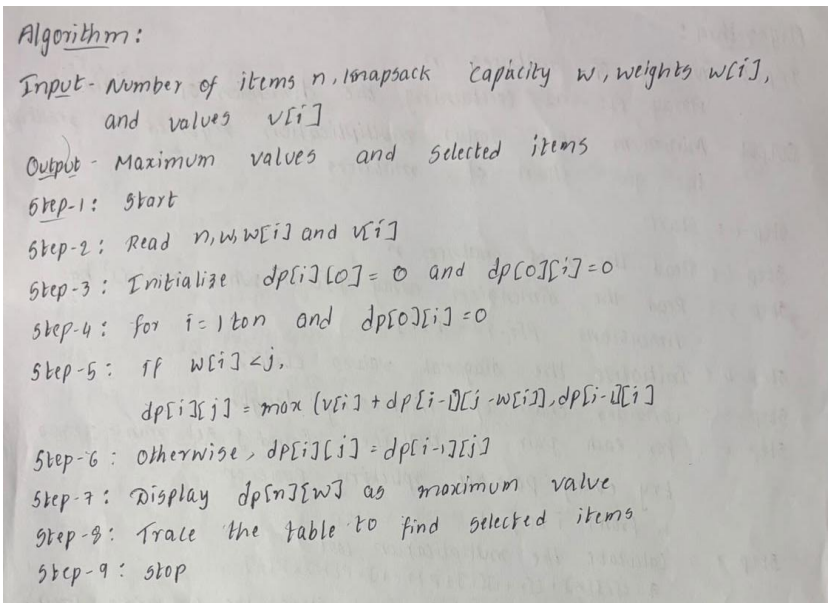
PRACTICAL - 5

AIM : Implementation of a knapsack problem using dynamic programming.

ALGORITHM:



PROGRAM:

```
#include <iostream>
#include <algorithm>
using namespace std;

class Knapsack {
private:
    int w[10];
    int v[10];
    int n;
    int W;
    int dp[10][100];

public:
    void getInput() {
        cout << "Enter number of items   : ";
```

```
    cin >> n;

    cout << "Enter knapsack capacity : ";
    cin >> W;

    for (int i = 1; i <= n; i++) {
        cout << "Item " << i << " weight and value : ";
        cin >> w[i] >> v[i];
    }
}

void solve() {
    for (int i = 0; i <= n; i++)
        dp[i][0] = 0;

    for (int j = 0; j <= W; j++)
        dp[0][j] = 0;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= W; j++) {
            if (w[i] <= j) {
                dp[i][j] = max(
                    v[i] + dp[i - 1][j - w[i]],
                    dp[i - 1][j]
                );
            } else {
                dp[i][j] = dp[i - 1][j];
            }
        }
    }
}

void printTable() {
    cout << "\nDP Table:" << endl;
    cout << "     ";
```

```
    for (int j = 0; j <= W; j++)
        cout << j << "  ";

    cout << endl;

    for (int i = 0; i <= n; i++) {
        cout << "i=" << i << "  ";

        for (int j = 0; j <= W; j++)
            cout << dp[i][j] << "  ";

        cout << endl;
    }
}

void findItems() {
    cout << "\nSelected Items:" << endl;
    int j = W;

    for (int i = n; i >= 1; i--) {
        if (dp[i][j] != dp[i - 1][j]) {
            cout << "Item " << i
                 << " (w=" << w[i]
                 << ", v=" << v[i]
                 << ") TAKEN" << endl;

            j = j - w[i];
        } else {
            cout << "Item " << i
                 << " (w=" << w[i]
                 << ", v=" << v[i]
                 << ") NOT taken" << endl;
        }
    }
}
```

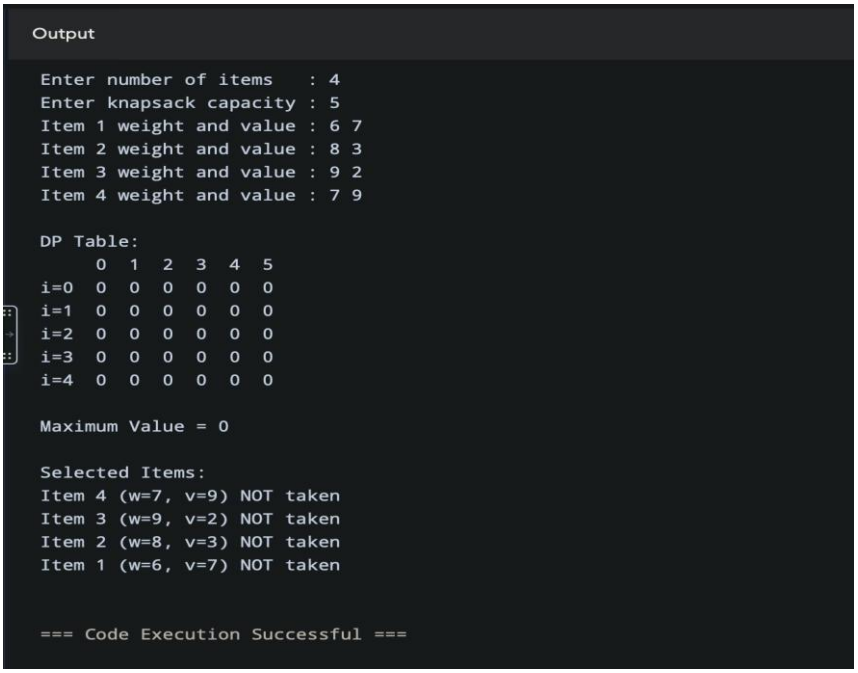
```
    void showResult() {
        cout << "\nMaximum Value = "
             << dp[n][W] << endl;
    }
};

int main() {
    Knapsack k;

    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();

    return 0;
}
```

OUTPUT:



```
Output

Enter number of items   : 4
Enter knapsack capacity : 5
Item 1 weight and value : 6 7
Item 2 weight and value : 8 3
Item 3 weight and value : 9 2
Item 4 weight and value : 7 9

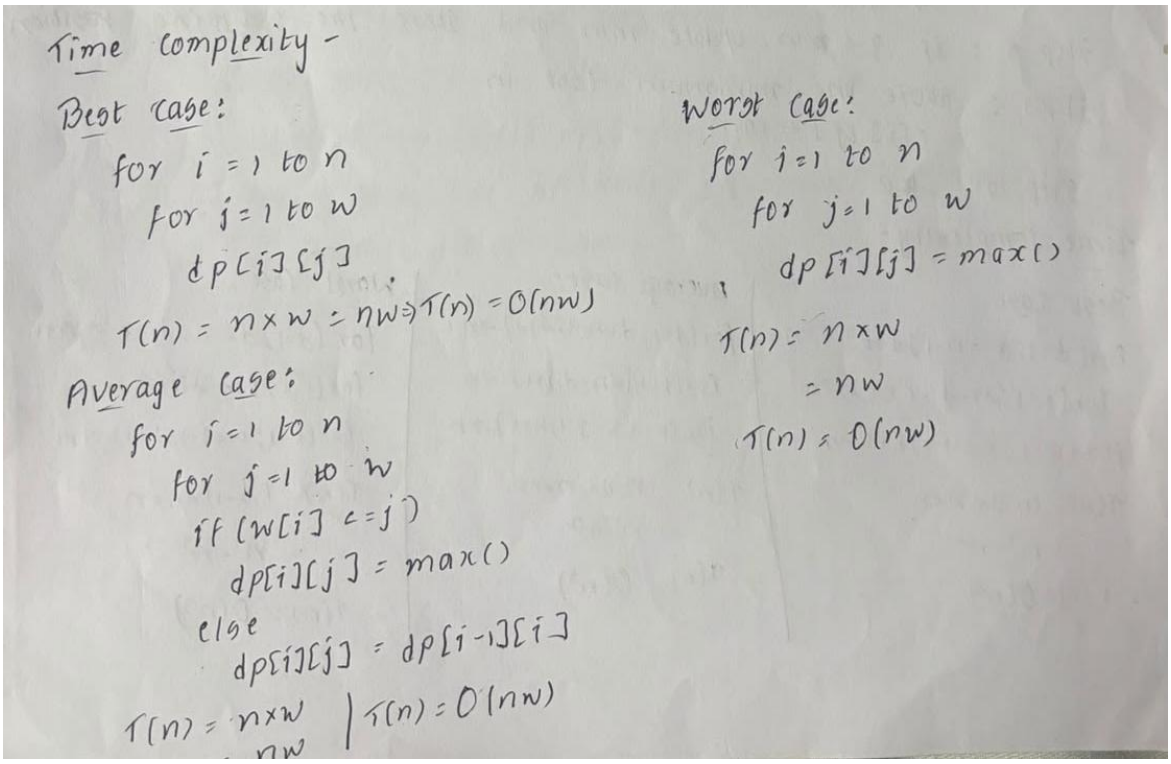
DP Table:
      0  1  2  3  4  5
i=0   0  0  0  0  0  0
i=1   0  0  0  0  0  0
i=2   0  0  0  0  0  0
i=3   0  0  0  0  0  0
i=4   0  0  0  0  0  0

Maximum Value = 0

Selected Items:
Item 4 (w=7, v=9) NOT taken
Item 3 (w=9, v=2) NOT taken
Item 2 (w=8, v=3) NOT taken
Item 1 (w=6, v=7) NOT taken


=== Code Execution Successful ===
```

TIME COMPLEXITY:



Conclusion:

The implementation and time analysis of 0/1 Knapsack using Dynamic Programming helps us understand its efficiency. The algorithm builds a DP table and considers whether to include or exclude each item to find the maximum value within the given capacity. It has a time complexity of O(nW) in the best, average, and worst cases. Therefore, Dynamic Programming provides an efficient and consistent solution for solving the 0/1 Knapsack problem